BLACK PEN Answer all questions first. GREEN PEN Then: tick and rewrite mistakes.

Review & Improve · GREEN PEN

Now change to GREEN PEN. Find each original question. Tick correct answers; read the model and rewrite mistakes.

Page 1 onwards · Do Now

Q1. An assembler.

Q2. It discards half the search space using an ordering comparison.

Q3. 1.

Q4. X is removed; Y remains at the front.

You Do pages · Independent questions

Q1. Two marks each: tokenizer recognises lexemes/tokens; parser validates grammar/structure; generator outputs machine/target code.

Q2. Identifier score, assignment =, identifier score, operator +, integer literal 1.

Q3. Identify potential redundant addition [2], valid only if semantics identical [2], exceptions/type/overflow/side effects or floating nuances [2], reasoned qualified evaluation [2].

Exit Ticket · Model answer

False. That is primarily lexical analysis.

Try the question again

Choose one question you got wrong. All correct? Choose the hardest.

1. Copy the whole original question:

2. Write a complete new answer in GREEN PEN. Show working if needed:

RHS COMPUTING / OCR H446 / mixed LESSON 115 · DRAFT

BLACK PEN Answer all questions first. GREEN PEN Then: tick and rewrite mistakes.

LESSON 115 · Lesson 115 · The Stages of Compilation

Name: _____ Class: _____ Date: _____

Work through the pages in order. Answer in BLACK PEN first. Check in GREEN PEN only after the Exit Ticket.

Do Now · Recall previous learning

Q1. What tool converts assembly mnemonics into machine instructions?

Q2. Why must a binary search operate on ordered data?

Q3. An OR gate has inputs 0 and 1. What is the output?

Q4. What is the effect of enqueue(X), enqueue(Y), dequeue() on an initially empty FIFO queue?

BLACK PEN Answer all questions first. **GREEN PEN** Then: tick and rewrite mistakes.

I Do · Read new knowledge

Key vocabulary

lexer — Recognises tokens from character stream.

parser — Checks tokens against grammar.

code generation — Production of target instructions.

optimisation — Meaning-preserving transformation to improve an objective.

New learning

Lexical analysis — Characters become tokens such as identifiers, numbers and operators.

Syntax analysis — A parser checks grammar and builds structural representation.

Code generation and optimisation — Target code is produced; valid transformations improve performance while preserving meaning.

Worked example

Explain the stages for the expression 'total = cost + 2' in a high-level source file.

1. Model answer — Lexer identifies identifier, assignment, identifier, plus, integer. Parser checks grammatical structure. The compiler creates suitable target instructions; optimisation might fold constants only where semantics permit. — Check: link each claim to the scenario and use precise OCR terminology.

We Do · Practise with guidance

A programmer writes 'total = (cost + 2' without a closing bracket. Which stage finds the fault?

What must we decide?

Identify the inputs, relevant facts and exact output or judgement required.

Work through the method

Use the middle stage of the visual to follow the process and state each intermediate result.

Shared worked answer

Lexical analysis can still recognise tokens. Syntax analysis detects a missing closing delimiter because token order violates grammar. No valid final executable follows until corrected.

BLACK PEN Answer all questions first. **GREEN PEN** Then: tick and rewrite mistakes.

You Do · Work independently

BLACK PEN: Write your own answers. Do not look at the Review & Improve pages yet.

Question 1

Describe lexical analysis, syntax analysis and code generation. [6]

Question 2

For 'score = score + 1', name five token categories or concrete tokens. [5]

Question 3

Evaluate whether an optimiser may replace 'x = a + 0' with 'x = a', considering language semantics. [8]

Exit Ticket · Answer before checking

A syntax analyser is responsible primarily for reading characters into tokens.